UNIVERSIDAD ANÁHUAC MAYAB

Ingeniería en Tecnologías de la Información y Negocios Digitales

Bitácora

Diario personal con fotos, dictado por voz y geolocalización

Reporte técnico · Parcial 2

Alumno	Marcos
Matrícula	
Profesor	
Tecnología	Flutter / Dart · SQLite
Fecha	Noviembre 2026

1. Justificación del proyecto

1.1 Problema

Llevar un diario personal tiene beneficios conocidos: ayuda a ordenar ideas, a reducir el estrés y a conservar recuerdos que con el tiempo se olvidan. Sin embargo, la mayoría de las personas abandona el hábito por tres razones prácticas:

- **Escribir toma tiempo.** Al final del día, teclear varios párrafos en el teléfono es lento y cansado.
- **El texto solo pierde contexto.** Meses después es difícil recordar *dónde* ocurrió algo o *cómo se veía*.
- **Las alternativas están dispersas.** Las fotos quedan en la galería, la ubicación en otra app y las notas en otra; nada está conectado.

1.2 Solución

Bitácora reúne en una sola entrada todo lo que describe un día: el texto, una foto y el lugar. Para que registrar sea rápido, aprovecha el hardware del teléfono:

Gadget	Cómo resuelve el problema
Micrófono + reconocimiento de voz	El usuario **dicta** su día en lugar de escribirlo; el texto aparece en tiempo real y se puede corregir a mano.
Cámara y galería	Una foto tomada al momento (o elegida de la galería) ilustra la entrada y ayuda a recordarla.
GPS + mapas	Al guardar, la app registra automáticamente la ubicación; en el detalle se muestran las coordenadas y un botón para abrirlas en Google Maps.

Toda la información se guarda **localmente en SQLite**: la app funciona sin conexión y los datos personales no salen del dispositivo, algo importante tratándose de un diario.

1.3 Usuarios y casos de uso

- Personas que quieren mantener un diario personal sin invertir mucho tiempo.
- Viajeros que desean una bitácora de viaje con fotos y mapa de cada lugar visitado.
- Estudiantes o profesionales que hacen trabajo de campo (visitas, inspecciones, prácticas) y necesitan notas con evidencia fotográfica y ubicación.

2. Funcionalidades

Pantalla	Qué hace	Requerimiento cubierto
Inicio (InicioScreen)	Lista de entradas, de la más nueva a la más vieja, con miniatura de la foto, fecha e ícono si tiene ubicación. Tiene menú lateral y botón “+”.	Scaffold, AppBar, Drawer, FutureBuilder, Hero
Nueva entrada (NuevaEntradaScreen)	Formulario: foto con cámara o galería, título y texto (escrito o dictado). Al guardar obtiene el GPS y guarda todo en SQLite.	Cámara, galería, micrófono, GPS, try-catch
Detalle (DetalleScreen)	Muestra la foto en grande (animación Hero), la fecha, el texto y las coordenadas, con un botón para abrirlas en Google Maps. Permite borrar la entrada.	Hero, Google Maps, try-catch

3. Arquitectura

3.1 Organización por capas

El código está dividido en carpetas según lo que hace cada parte. Las pantallas no hablan directo con SQLite ni con el hardware: le piden las cosas a una clase especializada.

Capa	Carpeta / archivo	Qué hace
Modelo	`models/entrada.dart`	Clase `Entrada`: los datos de un día (título, texto, fecha, foto, latitud y longitud) y cómo convertirlos para SQLite.
Datos	`database/db_helper.dart`	Crea la base de datos y tiene los métodos para insertar, leer y borrar entradas.
Servicios	`services/`	Una clase por gadget: `CamaraService`, `VozService` y `UbicacionService`.
Pantallas	`screens/`	Lo que ve el usuario: inicio, nueva entrada y detalle.
Arranque	`main.dart`	Crea la app, el tema y las rutas con nombre.

3.2 Principios SOLID aplicados

Principio	Cómo se aplica
S – Responsabilidad única	Cada clase hace una sola cosa: `DBHelper` sólo maneja la base de datos, `CamaraService` sólo las fotos, `VozService` sólo el micrófono y `UbicacionService` sólo el GPS.
O – Abierto/cerrado	Para agregar otro gadget se crea un servicio nuevo sin modificar los que ya existen.
D – Inversión de dependencias (parcial)	Las pantallas no usan directamente los paquetes de SQLite, cámara o GPS; usan los servicios. Si se cambia un paquete, sólo se modifica su servicio.

3.3 Diagrama de clases

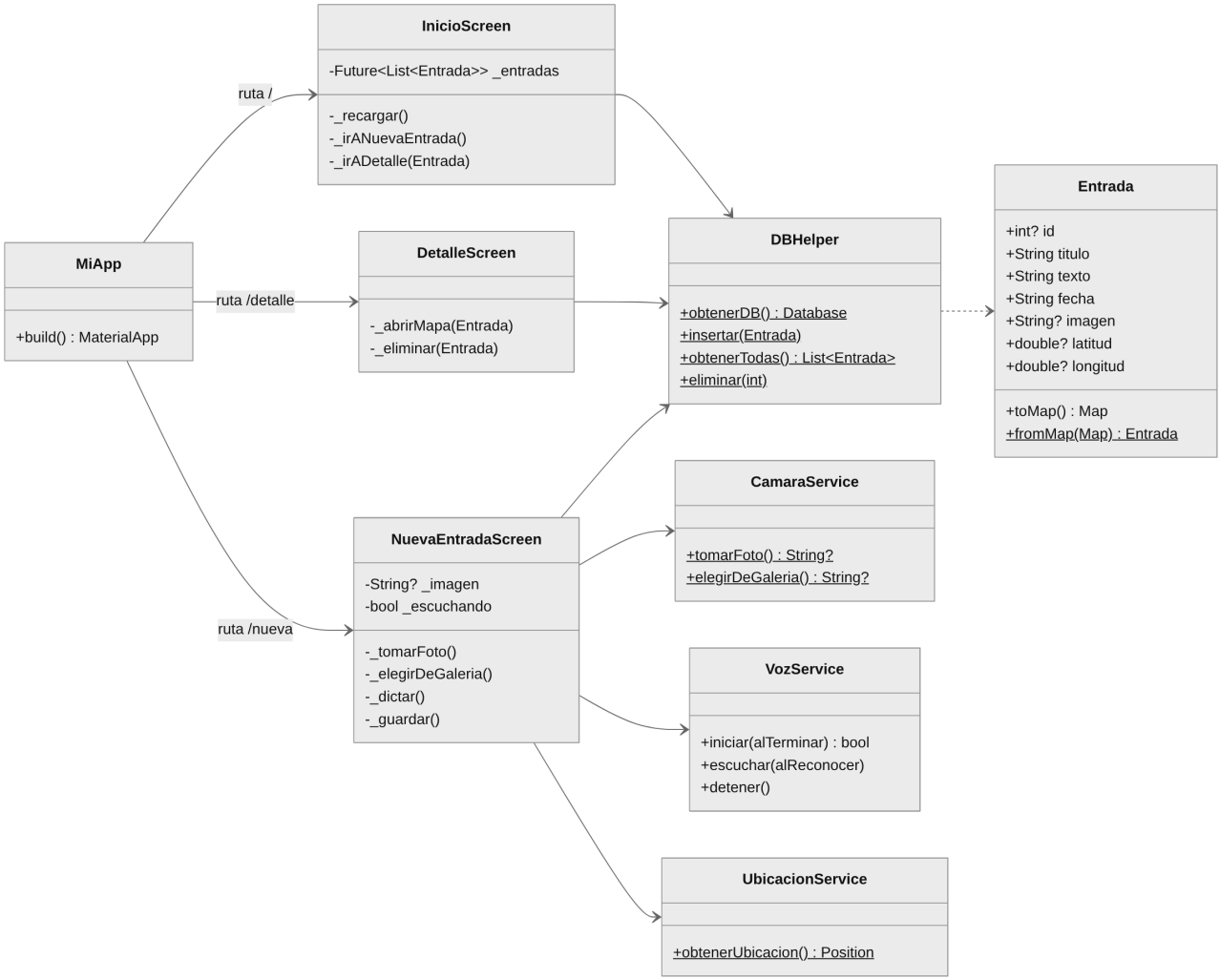


Figura 1. Clases de la aplicación y sus relaciones.

3.4 Flujo de datos

El hardware entrega los datos a los servicios; los servicios se los pasan ya listos al formulario (ruta de la foto, texto y coordenadas). El formulario arma una `Entrada` y la guarda con `DBHelper`. La pantalla de inicio lee las entradas con un `FutureBuilder` y, al tocar una, la manda como argumento a la pantalla de detalle.

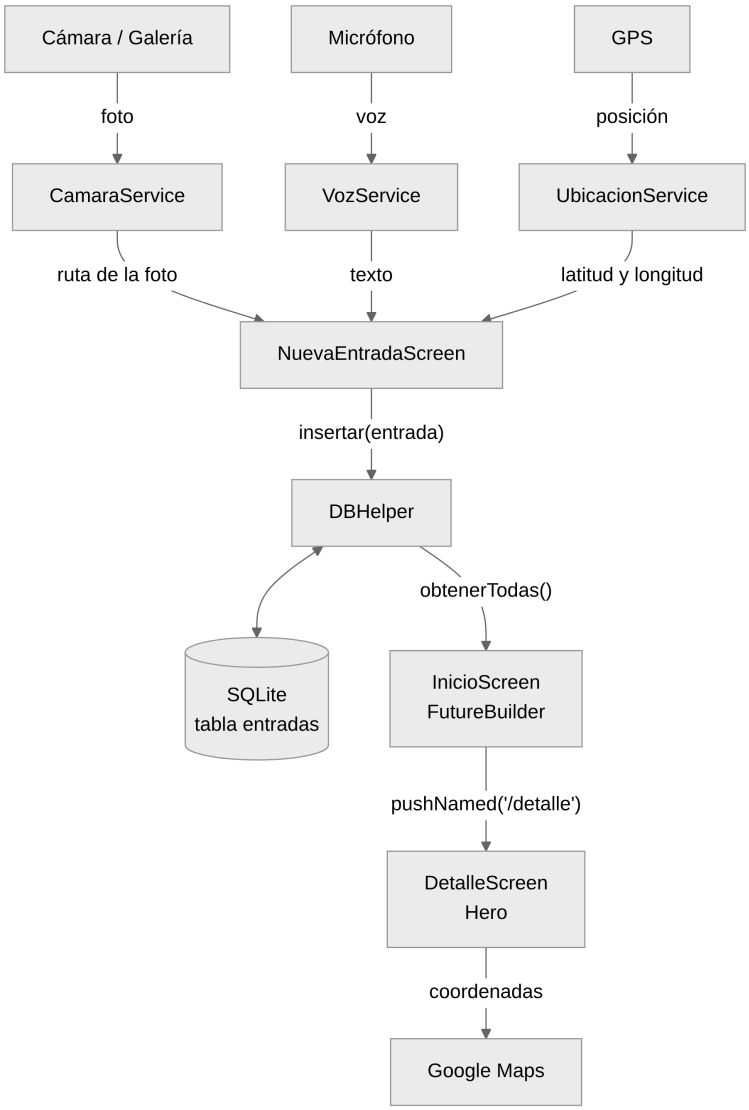


Figura 2. Flujo de datos de la aplicación.

3.5 Secuencia: crear una entrada

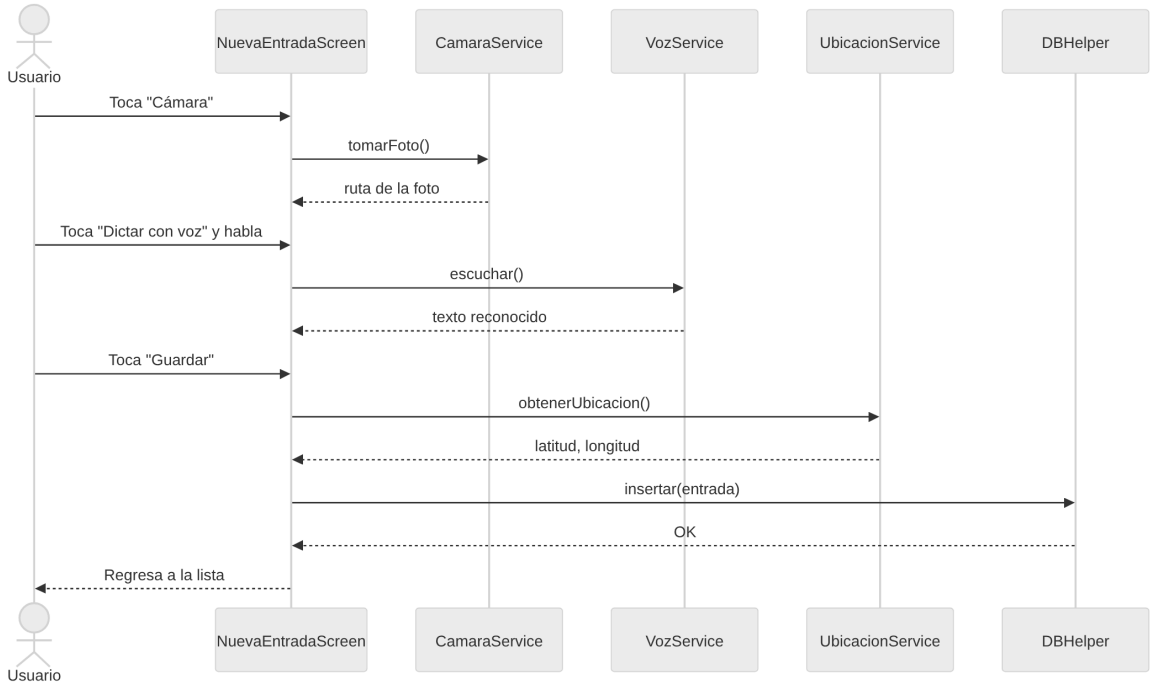


Figura 3. Pasos al crear y guardar una entrada.

4. Almacenamiento local

Se usó **SQLite** (paquete `sqflite`) porque cada entrada es un registro con varios campos y se necesitan listar ordenadas. SharedPreferences sirve para guardar valores sueltos (como una configuración), no listas de registros.

```
CREATE TABLE entradas(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  titulo TEXT,
  texto TEXT,
  fecha TEXT,
  imagen TEXT,     -- ruta de la foto en el teléfono
  latitud REAL,
  longitud REAL
)
```

La foto no se guarda dentro de la base de datos: se copia a la carpeta de la app y en SQLite sólo se guarda su **ruta**.

5. Manejo de errores y carga asíncrona

- **try-catch** al usar la cámara, la galería, el micrófono, el GPS, al guardar, al borrar y al abrir Google Maps. Si algo falla se muestra un mensaje (`SnackBar`) en lugar de que la app se cierre.
- Si el GPS está apagado o no hay permiso, `UbicacionService` lanza una excepción; el formulario la atrapa y **guarda la entrada sin ubicación** para no perder lo que escribió el usuario.
- **FutureBuilder** en la pantalla de inicio: muestra un indicador de carga mientras SQLite responde, un mensaje si hubo error, o la lista cuando llegan los datos.

6. Navegación y animación

En `main.dart` se definen **rutas con nombre**: `/` (inicio), `/nueva` (formulario) y `/detalle`. Para cambiar de pantalla se usa `Navigator.pushNamed`; para regresar, `Navigator.pop`. A la pantalla de detalle se le manda la entrada como `arguments`.

La animación **Hero** envuelve la foto en la lista y en el detalle con la misma etiqueta (`'foto' + id`), así Flutter anima la imagen de un lugar al otro al cambiar de pantalla.

7. Paquetes y permisos

Paquete	Para qué	Permiso Android
`image_picker`	Cámara y galería	`CAMERA`
`speech_to_text`	Micrófono: voz a texto	`RECORD_AUDIO`
`geolocator`	Ubicación GPS	`ACCESS_FINE_LOCATION`
`url_launcher`	Abrir Google Maps	`INTERNET`
`sqflite`, `path`, `path_provider`	Base de datos y carpeta de fotos	—

8. Conclusión

Bitácora cumple los requerimientos del parcial usando tres gadgets del teléfono (cámara, micrófono y GPS) en un flujo sencillo: tomar una foto, dictar lo que pasó y guardar. Separar el código en modelo, base de datos, servicios y pantallas hace que cada parte sea fácil de entender y de cambiar, y los bloques try-catch evitan que la app se cierre cuando falta un permiso o falla el GPS.